

机场航班运行仿真系统

需求分析说明书

Airport Flight Simulation System

项目	内容
项目类型	软件工程实验项目
技术路线	Java + Spring Boot + Maven + MySQL + Thymeleaf
文档版本	V1.0
编写日期	2026 年 6 月 5 日
适用阶段	需求分析与项目立项

修订记录

版本	日期	说明
V1.0	2026-06-05	根据项目规划整理形成正式需求分析说明书，补充业务场景、用户诉求、功能边界、开发范围与验收标准。

1 引言

1.1 编写目的

本文档用于明确“机场航班运行仿真系统”的业务背景、用户诉求、功能需求、非功能需求、数据需求、系统边界和验收标准，为后续概要设计、编码实现、测试验收和课程答辩提供依据。文档面向课程教师、项目组成员、测试人员以及后续维护人员，重点解决“系统要做什么、为什么要做、做到什么程度、哪些内容暂不做”的问题。

1.2 项目背景

机场航班运行具有典型的资源受限特征。跑道、停机位等机场核心资源数量有限，而航班按照计划时间持续产生起飞和降落请求。当多个航班在相近时间段竞争同一类资源时，机场需要按照调度规则安排航班顺序，并记录资源占用、航班等待和延误情况。本项目通过建立简化的机场运行模型，模拟航班从计划、等待、调度、起飞或降落、资源释放到结果统计的全过程。

本系统不是工业级空管系统，也不承担真实机场运行决策，而是面向软件工程课程实验，用于体现需求分析、数据库建模、业务流程建模、调度算法设计、Web 系统实现和测试验收等完整开发过程。

1.3 术语定义

术语	含义
航班	具有航班号、航空公司、机型、计划时间、起降类型和运行状态的仿真对象。
跑道	供航班起飞或降落使用的机场资源，可处于空闲、占用或维护状态。
停机位	供降落航班到达后停靠的机场资源，需与飞机机型匹配。
仿真任务	一次按指定开始时间、结束时间和时间步长执行的航班运行模拟过程。
等待队列	因到达计划时间或资源不足而等待调度的航班集合。
延误时间	航班因资源不足或排队等待导致实际执行时间晚于计划时间的分钟数。
FCFS	First Come First Served，先到先服务调度策略，基础版本采用该策略。

2 业务场景调研

2.1 业务现状抽象

从真实机场运行场景抽象来看，航班运行的核心矛盾是“计划航班请求持续到达”与“机场核心资源有限”之间的冲突。航班在计划起飞或降落时间附近需要使用跑道；降落航班还需要在落地后分配停机位。若资源被其他航班占用或处于维护状态，航班只能等待，并产生延误。对于课程项目而言，系统需要保留该业务矛盾的主要特征，同时对天气、空域、滑行路线、机组排班、旅客登机 etc 复杂因素做合理简化。

2.2 典型业务场景

场景	业务描述	系统应支持的能力
航班计划录入	管理员提前录入一天或一段时间内的起飞、降落航班计划。	支持航班新增、编辑、删除、查询和状态初始化。
机场资源配置	管理员维护跑道、停机位数量、状态和适用类型。	支持跑道和停机位基础数据管理，并为仿真调度提供资源状态。
航班集中到达	多个航班在相近时间请求跑道，资源数量不足。	系统将航班加入等待队列，按照规则调度，记录等待和延误。
降落资源约束	降落航班既需要跑道，也需要匹配机型的停机位。	调度时同时检查跑道和停机位，任一资源不足则继续等待。
仿真过程复盘	教师或学生需要查看每一步仿真事件，说明系统为何产生延误。	记录事件日志，展示状态变化、资源分配和延误原因。
结果分析展示	仿真结束后需要用于实验报告和课堂演示。	统计完成航班数、延误航班数、平均延误、最大延误和资源利用率。

2.3 核心业务痛点

- 航班计划、机场资源和仿真结果如果只通过人工表格维护，难以直观看出航班状态随时间变化的过程。
- 资源不足导致的等待和延误具有过程性，仅保存最终结果无法解释调度原因。
- 课程实验需要同时体现业务建模、数据库设计、算法逻辑、页面展示和测试验证，单一 CRUD 系统不足以体现软件工程综合能力。
- 调度算法若过于复杂，会超出课程周期；若过于简单，又无法体现资源竞争和延误统计价值，因此需要明确基础版和扩展版边界。

3 用户角色与核心诉求

角色	使用目标	核心诉求	关注点
系统管理员	维护航班、跑道、停机位和仿真任务。	快速录入基础数据，能够启动、推进、重置仿真。	数据正确性、操作便利性、异常提示。
课程教师/验收	检查项目是否符合软	看到清晰业务背景、完整功	需求完整性、边界清

人员	件工程实验要求。	能链路、可运行系统和可解释测试结果。	晰度、演示效果、文档规范性。
项目开发成员	根据需求完成设计、编码和测试。	需求边界明确，模块划分清晰，数据字段和业务规则可落地。	功能优先级、接口职责、数据库结构、测试依据。
普通查看用户（可选）	查看航班状态和统计结果。	无需修改数据，只需直观了解仿真结果。	页面可读性、状态展示、统计摘要。

3.1 用户核心诉求归纳

1. 能够把航班计划和机场资源录入系统，形成可持久化的基础数据。
2. 能够按时间步推进仿真，观察航班状态变化，而不是只得到静态列表。
3. 能够在资源不足时自动排队、累计延误，并记录延误原因。
4. 能够用清晰页面或接口展示跑道、停机位、航班状态、等待队列和统计结果。
5. 能够支撑课程提交材料，包括需求分析、概要设计、测试报告、运行截图和演示 PPT。

4 项目目标与开发范围

4.1 总体目标

开发一个基于 Java 的机场航班运行仿真系统，实现航班计划管理、机场资源管理、仿真任务控制、资源调度、延误统计和结果展示。系统应能够完整演示“基础数据录入—仿真启动—时间推进—资源分配—状态变化—日志记录—统计展示”的业务闭环。

4.2 本期开发范围

范围类别	纳入本期范围	暂不纳入或作为扩展
用户范围	管理员操作；普通查看用户可作为只读扩展。	复杂权限体系、角色审批、真实机场多岗位协同。
业务范围	航班起飞/降落计划、跑道分配、停机位分配、等待与延误统计。	旅客值机、安检、登机、行李、机组排班、空域流量控制。
资源范围	跑道、停机位两类核心资源。	滑行道冲突、登机口精细分配、维修车辆和机务资源。
算法范围	基础版采用 FCFS，并支持降落优先、计划时间优先等简单规则。	复杂优化算法、机器学习预测、多机场协同调度。
展示范围	表格页面、仿真日志、统计结果、仪表盘摘要。	三维机场可视化、地图轨迹动画、实时大屏系统。
部署范围	本地开发环境运行，MySQL 持久化。	云部署、高并发集群、真实外部航班数据接入。

4.3 成功标准

- 系统能够成功启动并连接 MySQL 数据库。
- 航班、跑道、停机位等基础数据能够新增、查询、修改、删除并持久化。
- 创建仿真任务后，航班状态能够随仿真时间推进发生变化。
- 当跑道或停机位资源不足时，系统能够让航班等待并累计延误。
- 系统能够保存仿真事件日志，并展示延误统计和资源利用结果。
- 需求、设计、测试等课程文档能够与系统实现保持一致。

5 功能需求

5.1 功能模块划分

模块编号	模块名称	主要职责	优先级
FR-01	航班信息管理	维护航班计划、机型、起降类型、计划时间、状态和延误信息。	必须
FR-02	机场资源管理	维护跑道和停机位信息，包括状态、适用类型和当前占用情况。	必须
FR-03	仿真任务管理	创建仿真任务，设置开始时间、结束时间、时间步长，支持启动、单步推进和重置。	必须
FR-04	调度算法	根据计划时间、航班类型和资源状态分配跑道与停机位。	必须
FR-05	仿真日志	记录航班进入队列、资源分配、延误、完成和资源释放等事件。	推荐
FR-06	统计展示	统计航班完成情况、延误情况、等待队列长度和资源利用率。	推荐
FR-07	系统管理	管理员登录、用户退出、操作日志等。	可选

5.2 航班信息管理需求

- 系统应支持新增航班，录入航班号、航空公司、出发机场、到达机场、起降类型、机型、计划时间和优先级。
- 系统应支持编辑航班计划信息，但已完成或取消航班的关键运行结果不应被随意覆盖。
- 系统应支持按航班号、航班状态、起降类型查询或筛选航班。
- 系统应显示航班当前状态、实际起飞/降落时间和延误分钟数。
- 系统应对航班号为空、计划时间为空、时间逻辑错误、重复航班号等情况进行校验和提示。

5.3 机场资源管理需求

- 系统应支持跑道新增、编辑、删除、查询，并记录跑道编号、状态、支持类型和当前占用航班。
- 系统应支持停机位新增、编辑、删除、查询，并记录停机位编号、状态、适用机型和当前占用航班。
- 跑道和停机位状态至少包括空闲、占用、维护三种。

- 处于维护状态的资源不得参与仿真调度。
- 资源编号应保持唯一，避免调度结果无法追踪。

5.4 仿真任务与时间推进需求

- 系统应支持创建仿真任务，设置任务名称、开始时间、结束时间和时间步长。
- 系统应采用离散时间步推进，例如每次推进 5 分钟或 10 分钟。
- 系统应在每个时间点检查计划时间小于等于当前仿真时间且未完成的航班，并加入等待队列。
- 系统应支持自动运行完整仿真，也应支持单步推进，便于课堂演示和调试。
- 系统应支持重置仿真，将航班状态、资源占用和统计数据恢复到初始状态。

5.5 调度算法需求

- 基础版本采用 FCFS 策略，即计划时间越早越优先。
- 当计划时间相同或相近时，降落航班优先于起飞航班，以体现降落航班对安全和停机资源的更强约束。
- 起飞航班调度时需检查是否存在可用跑道；若存在则分配跑道并更新航班状态，否则继续等待并增加延误。
- 降落航班调度时需同时检查可用跑道和匹配机型的可用停机位；任一资源不足则继续等待并增加延误。
- 系统应记录每次调度决策，包括分配资源、无法分配原因和延误时间累计。
- 增强版本可加入综合优先级分数，综合考虑基础优先级、延误时间权重、航班类型权重和紧急航班标记。

5.6 结果展示与统计需求

- 首页仪表盘应展示当前仿真时间、总航班数、已完成航班数、延误航班数、空闲跑道数和空闲停机位数。
- 仿真日志页面应展示事件时间、航班号、事件类型和事件描述。
- 统计结果页面应展示总航班数、起飞航班数、降落航班数、平均延误时间、最大延误时间、跑道利用率和停机位利用率。
- 航班列表应支持查看每个航班的计划时间、实际时间、当前状态和延误分钟数。

6 用例需求

用例编号	用例名称	参与者	前置条件	基本流程	后置结果
UC-01	维护航班计划	管理员	系统已启动。	进入航班管理页面，新增或编辑航班信息，提交保存。	航班数据保存到数据库，可被仿真任务读取。
UC-02	维护机场资源	管理员	系统已启动。	录入跑道和停机位，设置状态和适用类型。	资源信息保存到数据库，空闲资源可参与调度。
UC-03	创建仿真	管理员	已有航班和	设置任务名称、开始时	生成待运行的仿真

	任务		资源数据。	间、结束时间、时间步长并保存。	任务。
UC-04	启动仿真	管理员	仿真任务已创建。	点击启动或自动运行，系统按时间步推进。	航班状态、资源状态、日志和统计结果被更新。
UC-05	单步推进仿真	管理员/教师	仿真任务处于可运行状态。	每点击一次推进按钮，系统推进一个时间步。	便于观察每一步调度结果。
UC-06	查看仿真结果	管理员/查看用户	仿真已运行。	打开日志或统计页面查看事件和指标。	用户可了解延误原因和资源利用情况。

7 数据需求

7.1 核心数据实体

实体	关键字段	说明
flight	id, flight_no, airline, flight_type, aircraft_type, planned_departure_time, planned_arrival_time, actual_departure_time, actual_arrival_time, status, delay_minutes, priority	保存航班计划和运行结果，是仿真调度的核心对象。
runway	id, runway_no, status, support_type, current_flight_id, occupied_start_time, occupied_end_time	保存跑道基础信息和占用状态。
gate	id, gate_no, status, aircraft_type, current_flight_id	保存停机位信息和机型适配关系。
simulation_task	id, task_name, start_time, end_time, time_step_minutes, current_time, status	保存一次仿真任务的配置和运行进度。
simulation_record	id, task_id, flight_id, event_time, event_type, description	保存仿真过程事件，用于展示、追踪和测试验证。
user（可选）	id, username, password, role	当实现登录功能时使用。基础版本可不实现。

7.2 数据约束

- 航班号、跑道编号、停机位编号应具备唯一性。
- 仿真结束时间必须晚于开始时间，时间步长必须为正整数。
- 航班计划时间不能为空，起飞航班应具备计划起飞时间，降落航班应具备计划降落时间。

- 资源处于占用状态时，应记录当前占用航班；资源释放后应清空占用关系。
- 仿真记录应与仿真任务、航班建立关联，保证事件可追溯。

8 非功能需求

类别	需求描述
易用性	页面以表格和按钮为主，操作路径清晰，错误提示应明确指出字段和原因。
可靠性	基础数据校验应防止明显错误输入；仿真运行过程中不应出现资源重复分配。
可维护性	采用 Controller、Service、Repository/Mapper、Entity 分层结构，调度逻辑独立放置，便于测试和修改。
可测试性	调度算法、延误计算、资源分配和统计逻辑应可以通过单元测试或接口测试验证。
性能	课程实验数据规模下，支持数百条航班和几十个资源对象的仿真运行，页面响应保持可接受。
安全性	基础版本可不实现登录；若实现登录，应至少区分管理员和只读查看用户。
兼容性	系统在本地 JDK、Maven、MySQL 环境下运行，浏览器可访问 Thymeleaf 页面。

9 外部接口与运行环境需求

9.1 技术环境

- 开发语言：Java 17 或 Java 21。
- 后端框架：Spring Boot。
- 项目管理：Maven。
- 数据库：MySQL 8.x。
- 页面模板：Thymeleaf。
- 测试工具：JUnit 5，接口测试可使用 Apifox 或 Postman。
- 开发工具：VS Code。

9.2 页面与接口

系统基础版本以 Thymeleaf 页面为主要交互方式，可根据实现需要补充 REST 接口。页面至少包括首页仪表盘、航班管理、跑道管理、停机位管理、仿真控制、仿真日志和统计结果。接口或页面操作应调用同一套 Service 业务逻辑，避免页面展示与仿真规则不一致。

10 功能边界与约束条件

10.1 明确纳入范围

- 航班计划管理、跑道管理、停机位管理。
- 按离散时间步推进的仿真任务。
- 跑道和停机位资源分配。
- 等待队列、延误累计、仿真日志记录。
- 统计指标和页面展示。
- MySQL 数据持久化。

10.2 明确不纳入范围

- 不接入真实民航运行数据，不承担真实航班调度决策。
- 不模拟复杂天气、空域流控、滑行路径冲突、旅客登机、行李和机组排班。
- 不要求高并发、多机场协同、分布式部署和工业级容灾。
- 不强制实现用户登录；登录、图表、天气影响和多算法对比作为增强功能。

10.3 约束条件

- 项目周期受课程实验安排限制，应优先完成基础版功能。
- 调度算法应先保证正确、可解释、可测试，再考虑复杂优化。
- 页面应优先保证可用和可演示，不追求复杂前端框架。
- 数据库表结构应围绕 flight、runway、gate、simulation_task、simulation_record 五类核心表设计。

11 验收标准

验收项	验收标准
基础数据管理	能够完成航班、跑道、停机位的新增、查询、修改和删除，数据能保存到 MySQL。
仿真任务	能够创建任务并设置开始时间、结束时间和时间步长。
状态流转	启动仿真后，航班能从计划状态进入等待、运行、完成或延误等状态。
资源调度	系统能够根据资源状态分配跑道和停机位，维护状态下的资源不会被分配。
延误统计	资源不足时航班继续等待，延误时间随时间步累计。
日志记录	系统能够记录进入队列、分配资源、延误、完成、释放资源等事件。
结果展示	页面能够展示航班状态、资源状态、仿真日志和统计结果。
测试材料	提供功能测试、异常测试和核心算法测试用例，测试结果与需求一致。
文档一致性	需求分析、概要设计、测试报告与实际实现的功能范围保持一

	致。
--	----

12 风险分析与控制措施

风险	影响	控制措施
功能范围过大	导致核心仿真功能无法按时完成。	优先完成基础版，将天气、登录、图表、多算法对比放入扩展功能。
调度算法过复杂	实现和测试难度增加。	先实现 FCFS 和简单优先规则，后续再扩展综合优先级。
页面开发耗时	影响业务逻辑实现和测试。	采用 Thymeleaf 表格页面，先可用后美化。
数据库设计反复修改	造成代码、SQL、文档不一致。	先固定五类核心表，字段以课程需求为主，避免过度设计。
测试数据不足	无法体现等待和延误场景。	设计多航班少资源、降落无停机位、资源维护等典型测试数据。

13 需求优先级

优先级	功能内容
必须完成	航班管理、跑道管理、停机位管理、仿真任务创建、仿真启动、航班状态变化、跑道分配、停机位分配、延误统计、MySQL 持久化。
推荐完成	页面可视化、等待队列展示、资源占用状态展示、仿真日志、统计结果页面、基础测试用例。
有时间再做	天气影响、紧急航班优先、多跑道策略、图表展示、用户登录、操作日志、不同调度算法对比。

14 附录：业务流程摘要

14.1 总体流程

1. 管理员录入机场跑道和停机位资源。
2. 管理员录入航班计划。
3. 管理员创建仿真任务并设置时间范围和步长。
4. 系统按时间步推进仿真。
5. 系统检查到达计划时间的航班并加入等待队列。
6. 调度算法根据规则分配跑道和停机位。
7. 系统更新航班状态和资源状态，并写入仿真事件。

8. 仿真结束后系统生成统计结果。

14.2 起飞调度流程

1. 获取等待起飞航班并按计划起飞时间排序。
2. 检查是否存在支持起飞且处于空闲状态的跑道。
3. 若存在可用跑道，分配跑道，更新航班状态为起飞中或完成，记录实际起飞时间。
4. 若不存在可用跑道，航班继续等待并增加延误时间。

14.3 降落调度流程

1. 获取等待降落航班并按计划降落时间排序。
2. 检查是否存在支持降落且空闲的跑道。
3. 检查是否存在匹配机型且空闲的停机位。
4. 若跑道和停机位均可用，分配资源，更新航班状态并记录实际降落时间。
5. 若任一资源不可用，航班继续等待并增加延误时间。